PROCESS MANAGEMENT COM3306

H.M.C.J.DE SILVA

DEPARTMENT OF COMPUTING

RAJARATA UNIVERSITY OF SRI LANKA

LEARNING OBJECTIVES

- At the end of the chapter student should be able to:
  - Define what a process is
  - Understand process states and transitions
  - Explain Process Control Block (PCB)
  - Understand scheduling and types of schedulers
  - Describe context switching
  - Differentiate between threads and processes

PROCESS

- The program in execution is called a process. In order to accomplish its task process needs the computer resources like CPU , memory and I/O devices.
- Differences between process and Program

Process	Program
Process is a dynamic object	Program is a static object
Process is a sequence of instruction execution	Program is a sequence of instructions
Process loaded into main memory	Program loaded into secondary storage devices
Time span of process is limited	Time span of program is unlimited

ATTRIBUTED OF A PROCESS

- Process ID- When a process is created a unique ID is assigned to the process for unique identification of the process in the system.
- Program counter-stores address of the last instruction of the process on which the process was suspended.
- Process state- process from its creation to its completion goes through various states: new, ready, running, and waiting.
- Priority- The process with the highest priority among the processes gets the CPU first. This is also stored on the process control block.
- General purpose registers- Every process has its own set of registers which are used to hold the data which is generated during the execution of the process.

ATTRIBUTED OF A PROCESS

- List of open files- During the execution, every process uses some files which need to be present in the main memory. OS also maintains a list of open files in the Process Control Block(PCB).
- List of Open devices- OS also maintains a list of open devices used during the execution process.

PROCESS STATES

- The process from its creation to completion passes through various states. Minimum number of states is five.

1. New

A program which is going to be picked up by the OS into the main memory is called a new process

2. Ready

Whenever a process is created it directly enters to ready state, in which it waits for the CPU to be assigned. There can be many processes present in the ready state.

PROCESS STATES

3. Running

One of the processes will be chosen from the ready state by OS depending on the scheduling algorithm.

If we have only one CPU in our system, the number of running processes for a particular time will always be one. If we have n processors in the system then we can have n processes running simultaneously.

4. Block or wait

From running state process can make transitions to block or wait state.

When a process waits for a certain resource to be assigned or for input from the user then the OS moves this process to the block or wait state and assigns the CPU to the other processes.

PROCESS STATES

5. Completion or termination

When the process finishes its execution, it comes in the termination state. All the context of the process will be deleted when the process is terminated by the OS.

Sub States

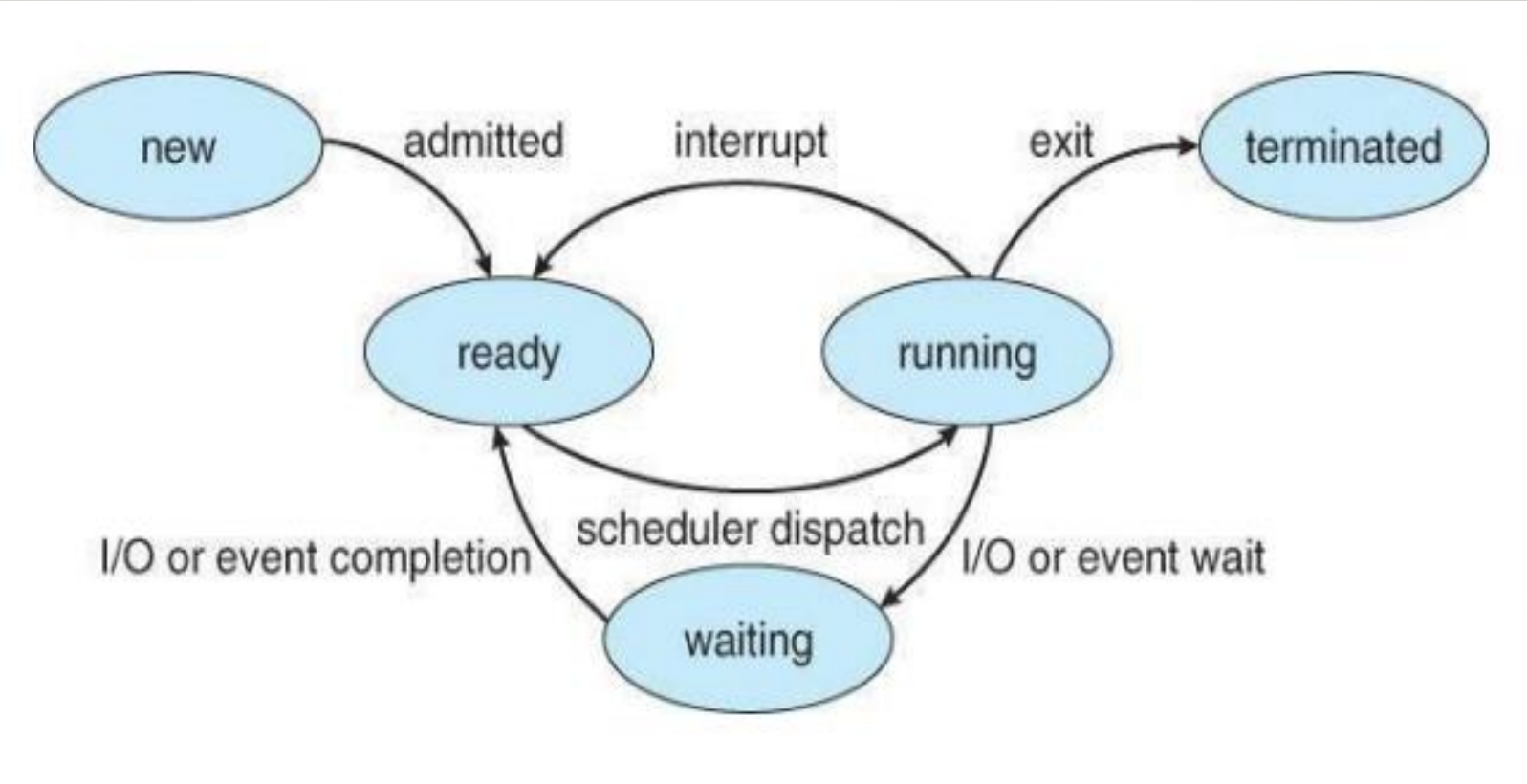
6. Suspend ready

A process in the ready state, which is moved to secondary memory from the main memory due to lack of resources (mainly primary memory) is called in the suspend ready state.

7. Suspend wait

Instead of removing the process from the ready queue, it's better to remove the blocked process which is waiting for some resources in the main memory.

PROCESS STATES



PROCESS STATES

- New->Ready : OS creates a process and prepares the process to be executed, then OS moves the process into the ready queue.
- Ready->Running : OS selects one of the Jobs from ready Queue and move them from ready to Running.
- Running->Terminate : When the Execution of a process has been Completed, OS terminates that process from running state
- Running->Ready : When the time slot of the processor expired (or) If the processor received any interrupt signal, the OS shifted Running -> Ready State

PROCESS STATES

- Running -> Waiting: A process is put into the waiting state if the process needs an event occurs (or) an I/O Device require,
- Waiting->Ready: A process in the waiting state is moved to the ready state when the event for which it has been Completed.

CONTEXT SWITCHING

- Context switching is the process of storing the state of a currently running process and restoring the state of the next scheduled process so that execution can resume from the same point later.

Why it's needed:

- To enable **multitasking**
- To allow the OS to switch between **multiple processes or threads**
- Ensures **fair CPU time** for all processes

WHEN DOES CONTEXT SWITCHING OCCUR?

- On a time slice expiration (in preemptive multitasking)
- When a higher priority process becomes ready
- During I/O wait or blocking
- On a system call like sleep() or wait()

STEPS INVOLVED IN CONTEXT SWITCHING

1. Save the context (state) of current process : CPU registers, Program counter, stack pointer, Other PCB details
2. Update the PCB of the current process to mark its state (e.g., from Running to Ready or Waiting)
3. Select a new process from the ready queue using a scheduler
4. Load the context of the selected process from its PCB
5. Resume execution of the new process

OVERHEAD OF CONTEXT SWITCHING

- Time-consuming : Involves saving/restoring data
- Doesn't do useful work directly : It's OS bookkeeping
- More switches = more overhead
- Minimized using threads or lightweight processes

OPERATIONS ON THE PROCESS

- Creation
- Scheduling
- Execution
- Deletion/Killing

WHAT IS PROCESS SCHEDULING?

- Process scheduling is the OS mechanism that decides which process gets the CPU and for how long, when multiple processes are ready to run.
- Goal: Efficient CPU utilization, fairness, responsiveness, meeting performance criteria(like throughput, turnaround time, etc.)
- Why is scheduling important?
  - Enables multiprogramming
  - Keeps the CPU busy
  - Ensures fairness among processes
  - Improves system performance

PROCESS SCHEDULING IN OS

- Long term scheduler
- Short term scheduler
- Medium term scheduler

LONG TERM SCHEDULER

- Also known as job scheduler.
- It chooses the processes from the secondary memory and keeps them in the ready queue maintained in the primary memory.
- The purpose of the long-term scheduler is to choose a perfect mix of IO-bound and CPU-bound processes among the jobs present in the pool.
- If the job scheduler chooses more IO bound processes, then all of the jobs may reside in the blocked state all the time and the CPU will remain idle most of the time.
- This will reduce the degree of Multiprogramming.
- Therefore, the Job of long-term scheduler is very critical and may affect the system for a very long time.

SHORT TERM SCHEDULER

- Also known as CPU scheduler.
- It selects one of the Jobs from the ready queue and dispatch to the CPU for the execution.
- A scheduling algorithm is used to select which job is going to be dispatched for the execution.
- The Job of the short term scheduler can be very critical in the sense that if it selects job whose CPU burst time is very high then all the jobs after that, will have to wait in the ready queue for a very long time.
- This problem is called starvation which may arise if the short term scheduler makes some mistakes while selecting the job.

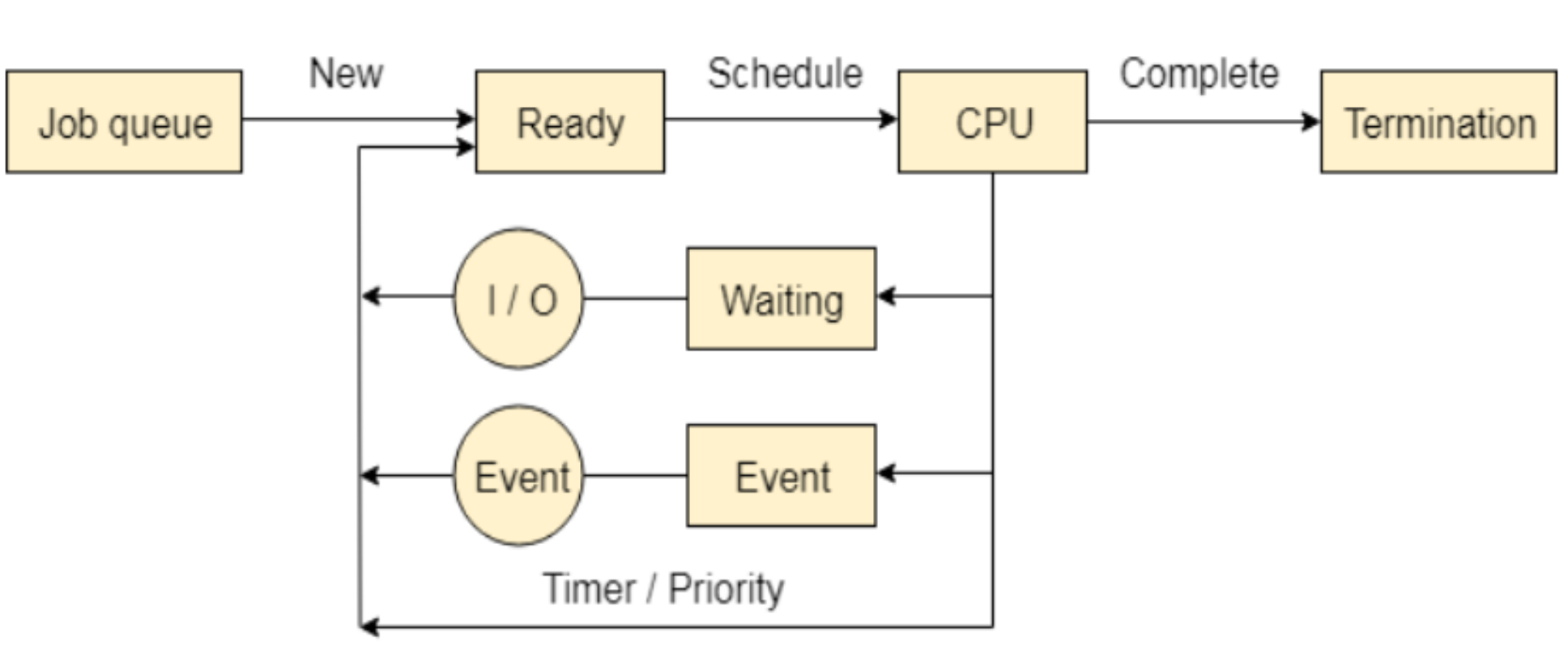
MEDIUM-TERM SCHEDULER

- Take care of swapped-out processes.
- If running state process needs IO time then the state should change from running to waiting.
- It removes the process from the running state to make room for the other processes.
- Such processes are the swapped-out processes and this procedure is called swapping.
- The medium-term scheduler is responsible for suspending and resuming the processes.
- The swapping is necessary to have a perfect mix of processes in the ready queue.

PROCESS QUEUES

- Scheduling queues are data structures used by the OS to manage process states and determine scheduling decisions.
- Each process moves through different queue during its lifecycle based on current state.
- OS manages various types of queues for each of the process state.
- The PCB related to the process is also stored in the queue of the same state.
- If the Process is moved from one state to another state then its PCB is also unlinked from the corresponding queue and added to the other state queue in which the transition is made.

PROCESS QUEUES



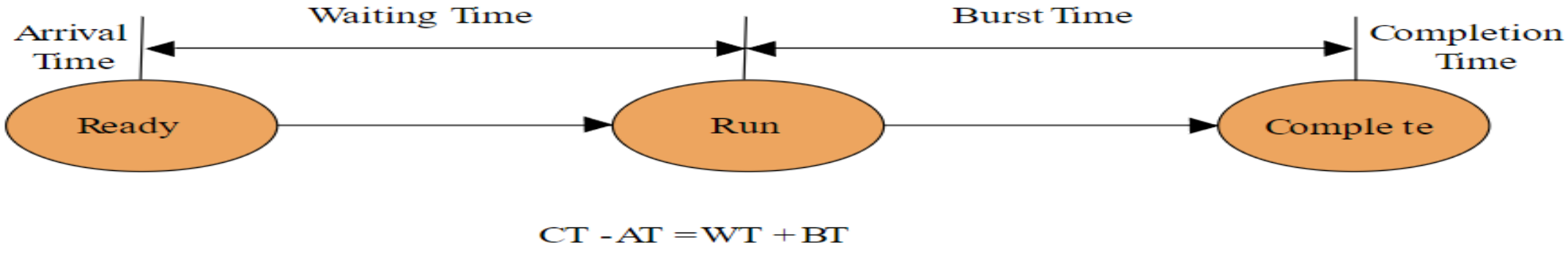
PROCESS QUEUES

1. Job Queue-In starting, all the processes get stored in the job queue. It is maintained in the secondary memory. The long term scheduler (Job scheduler) picks some of the jobs and put them in the primary memory.
2. Ready Queue-Ready queue is maintained in primary memory. The short term scheduler picks the job from the ready queue and dispatch to the CPU for the execution.
3. Waiting Queue-When the process needs some IO operation in order to complete its execution, OS changes the state of the process from running to waiting. The context (PCB) associated with the process gets stored on the waiting queue which will be used by the Processor when the process finishes the IO.

TIME RELATED TO PROCESS

- **Arrival time-** Time at which process enters into ready queue is called arrival time.
- **Burst time-**The total amount of time required by the CPU to execute the whole process is called the Burst Time. This does not include the waiting time
- **Completion time**-The Time at which the process completes its execution, is called completion time.
- **Turnaround time**-The total amount of time spent by the process from its arrival to its completion.
- **Waiting time**-The Total amount of time for which the process waits for the CPU to be assigned is called waiting time.
- **Response time-**The difference between the arrival time and the time at which the process first gets the CPU is called Response Time.

TIME RELATED TO PROCESS



$$CT - AT = WT + BT$$

$$TAT = CT - AT$$

$$\text{Waiting Time} = TAT - BT$$

TAT → Turn around time

BT → Burst time

AT → Arrival time

CPU SCHEDULING

- **In Multiprogramming systems**, the Operating system schedules the processes on the CPU to have the maximum utilization of it and this procedure is called **CPU scheduling**.
- The Operating System uses various scheduling algorithm to schedule the processes.
- This is a task of the short term scheduler to schedule the CPU for the number of processes present in the Job Pool.
- Whenever the running process requests some IO operation then the short term scheduler saves the current context of the process (also called PCB) and changes its state from running to waiting.

CPU SCHEDULING

- During the time, process is in waiting state; the Short term scheduler picks another process from the ready queue and assigns the CPU to this process. This procedure is called **context switching.**
- The Operating system maintains a process control block during the lifetime of the process.
- The Process control block is deleted when the process is terminated or killed.

SCHEDULING ALGORITHMS IN OS

- Purpose of scheduling algorithms:
  - Maximum CPU utilization
  - Fare allocation of CPU
  - Maximum throughput
  - Minimum turnaround time
  - Minimum waiting time
  - Minimum response time

TYPES OF SCHEDULING ALGORITHMS IN OS

- First Come First Serve-The process with the minimal arrival time will get the CPU first. The lesser the arrival time, the sooner the process gets the CPU. It is the non-preemptive type of scheduling.
- **Round Robin** –OS define time quantum(slice).All processes will get executed in cyclic way. Each of the process will get the CPU for a small amount of time (called time quantum) and then get back to the ready queue to wait for its next turn. It is a preemptive type of scheduling.
- **Shortest Job First**-The job with the shortest burst time will get the CPU first.
- **Shortest remaining time first-** The OS schedules the Job according to the remaining time of the execution.

TYPES OF SCHEDULING ALGORITHMS IN OS

- Priority based scheduling-The priority will be assigned to each of the processes. The higher the priority, the sooner will the process get the CPU. If the priority of the two processes is same then they will be scheduled according to their arrival time.
- **Highest Response Ratio Next**-The process with highest response ratio will be scheduled next. This reduces the starvation in the system.

PREEMPTIVE VS NON PREEMPTIVE APPROACH

- **Preemptive approach-**OS allocates the resources to a Process for a predetermined period. The process transitions from running state to ready state or from waiting state to ready state during resource allocation. This switching happens because the CPU may assign other processes precedence and substitute the currently active process for the higher priority process.
- **Non-preemptive approach**-The resource cannot be withdrawn from a process before the process has finished running. If a process or job is started execution, then the operating system must complete its process or job. Until the process or job is zero the new or next process or job does not start its execution.

FIRST COME FIRST SERVE

- In FCFS scheduling, the process that arrives first in the ready queue is allocated the CPU first.
- Processes are executed strictly in the order of their arrival time.
- FCFS supports non-preemptive CPU scheduling algorithms.

Features are:

1. In order to allocate processes, it uses the First In First Out queue.
2. The FCFS CPU Scheduling Process is straight forward and easy to implement.
3. As there is no consideration of process priority, it is an equitable algorithm.

FIRST COME FIRST SERVE

- Disadvantages

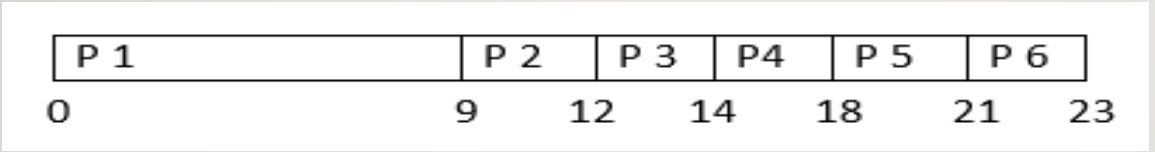
1. FCFS non-preemptive CPU Scheduling Algorithm has Long Waiting Time
2. FCFS CPU Scheduling favors CPU over Input or Output operations
3. In FCFS there is a chance of occurrence of Convoy Effect
4. Because FCFS is so straight forward, it often isn't very effective. Extended waiting periods go hand in hand with this. All other orders are left idle if the CPU is busy processing one time-consuming order.

FIRST COME FIRST SERVE

Process ID	Arrival Time	Burst Time
P1	0	9
P2	1	3
P3	1	2
P4	1	4
P5	2	3
P6	3	2

FIRST COME FIRST SERVE-NON PREEMPTIVE APPROACH

- Gantt chart for above example



Turn Around Time=Completion Time-Arrival time

Waiting time=Turn around time- Burst Time

FIRST COME FIRST SERVE

Process ID	Arrival Time	Burst Time	Completion Time	Turn around Time	Waiting Time
P1	0	9	9	9	0
P2	1	3	12	11	8
P3	1	2	14	13	11
P4	1	4	18	17	13
P5	2	3	21	19	16
P6	3	2	23	20	18

FIRST COME FIRST SERVE

- Average CT=(9+12+14+18+21+23)/6
  =16.6667
- Average WT=(0+8+11+13+16+18)/6
  =66/6
  =11
- Average TAT=(9+11+13+17+19+20)/6
  =89/6
  =14.83

SHORTEST JOB FIRST SCHEDULING

- This is a non preemptive scheduling algorithm.
- SJF scheduling algorithm, schedules the processes according to their burst time .Selects the process with smallest burst time.
- If we face a condition where two processes are present in the Ready Queue and their Burst Times are same, we can choose first mentioned process for execution.
- In actual Operating Systems, if we face the same problem then sequential allocation of resources takes place.

Advantages

- Maximum throughput
- Minimum average waiting

SHORTEST JOB FIRST SCHEDULING

Disadvantages

1. May suffer with the problem of starvation

Question

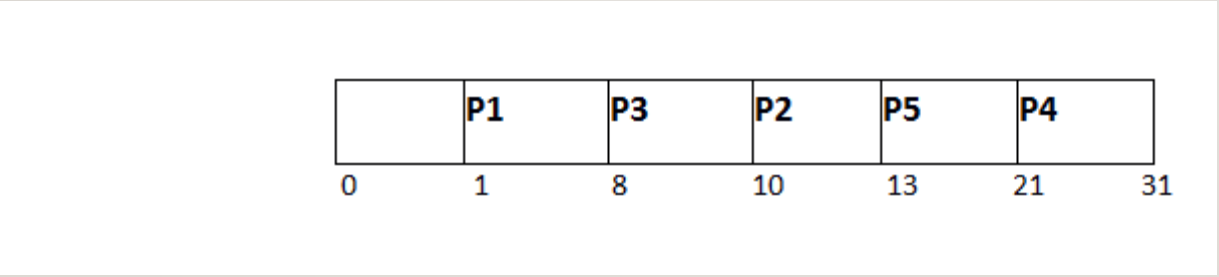
PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8

SHORTEST JOB FIRST SCHEDULING

PID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	1	7	8	7	0
2	3	3	13	10	7
3	6	2	10	4	2
4	7	10	31	24	14
5	9	8	21	12	4

SHORTEST JOB FIRST SCHEDULING

- Since, No Process arrives at time 0 hence; there will be an empty slot in the **Gantt chart** from time 0 to 1 (the time at which the first process arrives).
- Till now, we have only one process in the ready queue hence the scheduler will schedule this to the processor no matter what is its burst time.



SHORTEST REMAINING TIME FIRST(SRTF) SCHEDULING

- This Algorithm is the preemptive version of SJF scheduling.
- At the arrival of every process, the short term scheduler schedules the process with the least remaining burst time among the list of available processes and the running process.
- Once all the processes are available in the ready queue, No preemption will be done and the algorithm will work as SJF scheduling.
- The context of the process is saved in the Process Control Block when the process is removed from the execution and the next process is scheduled.
- This PCB is accessed on the next execution of this process.

SHORTEST REMAINING TIME FIRST(SRTF) SCHEDULING

- **Advantages of SJF**
- Lowest average waiting time
- Efficient CPU utilization

- **Disadvantages of SJF**
- Difficult to predict burst time accurately
- Starvation of long processes
- Not suitable for interactive systems

SHORTEST REMAINING TIME FIRST(SRTF) SCHEDULING

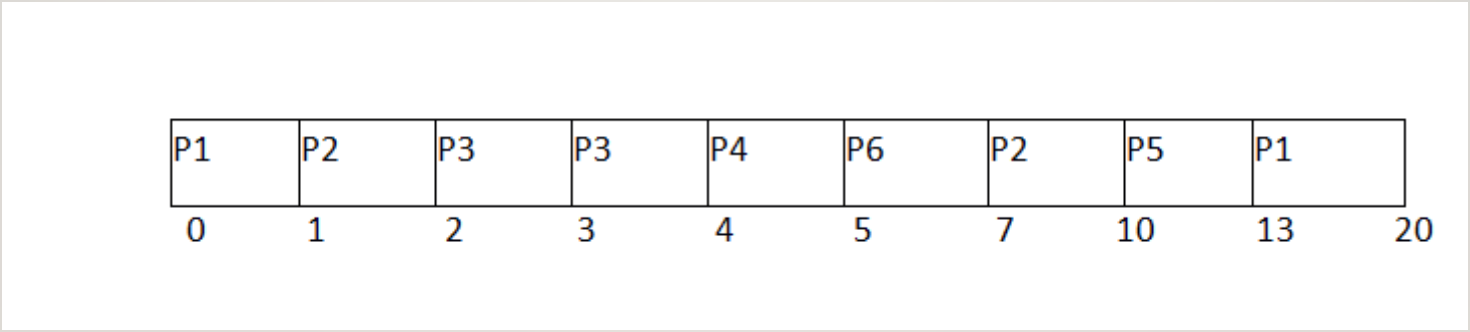
Process ID	Arrival Time	Burst Time
1	0	8
2	1	4
3	2	2
4	3	1
5	4	3
6	5	2

SHORTEST REMAINING TIME FIRST(SRTF) SCHEDULING

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time	Response Time
1	0	8	20	20	12	0
2	1	4	10	9	5	1
3	2	2	4	2	0	2
4	3	1	5	2	1	4
5	4	3	13	9	6	10
6	5	2	7	2	0	5

SHORTEST REMAINING TIME FIRST(SRTF) SCHEDULING

- Avg waiting time=24/6



- Once all the processes arrive, No preemption is done and the algorithm will work as SJF.

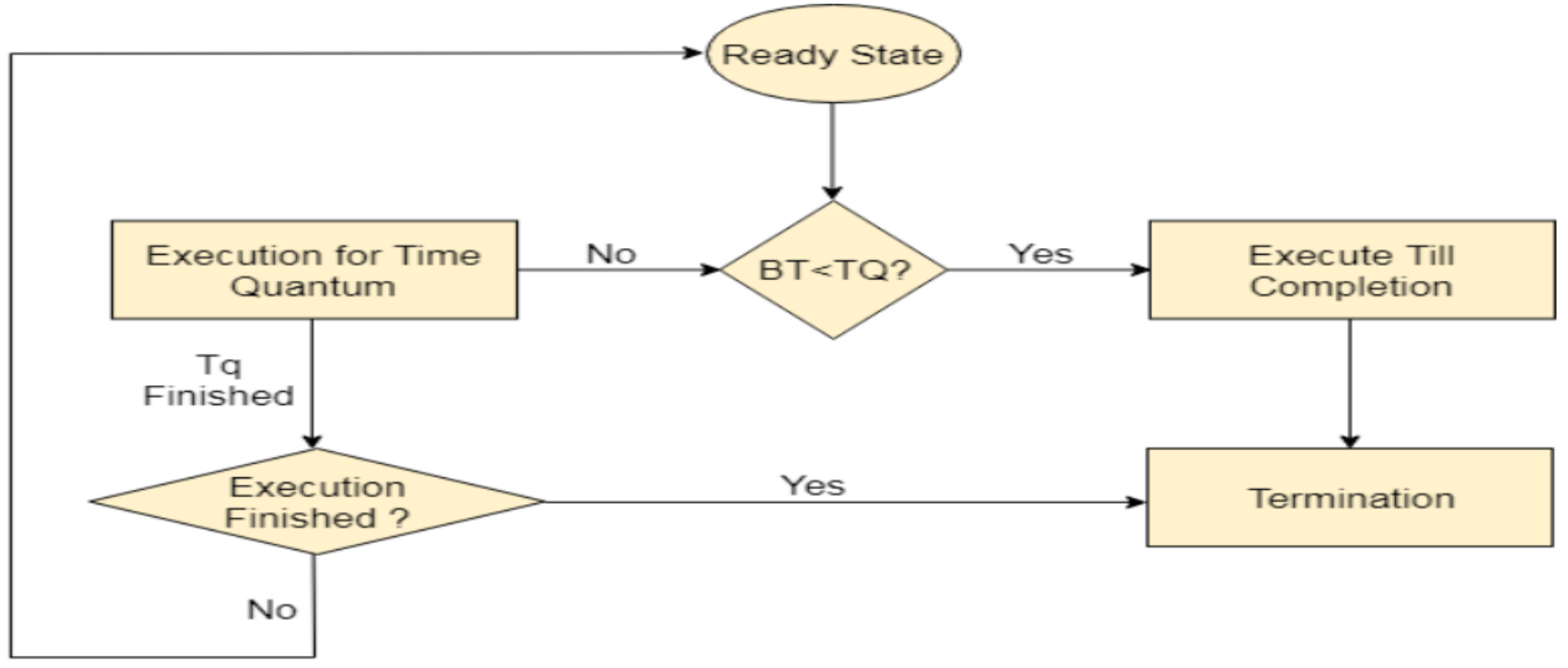
ROUND ROBIN SCHEDULING

- Round Robin CPU Scheduling uses Time Quantum (TQ).
- The Time Quantum is something which is removed from the Burst Time and lets the chunk of process to be completed.
- Time Sharing is the main emphasis of the algorithm. Each step of this algorithm is carried out cyclically. The system defines a specific time slice, known as a time quantum.
- First, the processes which are eligible to enter the ready queue enter the ready queue. After entering the first process in Ready Queue is executed for a Time Quantum chunk of time.

ROUND ROBIN SCHEDULING

- After execution is complete, the process is removed from the ready queue. Even now the process requires some time to complete its execution, then the process is added to Ready Queue.
- The Ready Queue does not hold processes which already present in the Ready Queue. The Ready Queue is designed in such a manner that it does not hold non unique processes.
- By holding same processes Redundancy of the processes increases.

ROUND ROBIN SCHEDULING

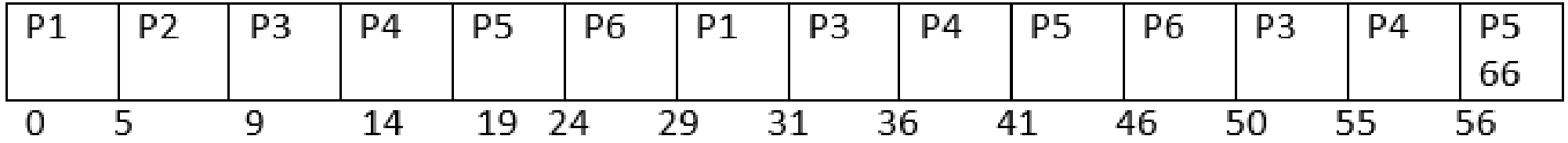


ROUND ROBIN SCHEDULING

S NO	Process ID	Arrival time	Burst Time
1	P1	0	7
2	P2	1	4
3	P3	2	15
4	P4	3	11
5	P5	4	20
6	P6	4	9

Assume time quantum TQ=5

ROUND ROBIN SCHEDULING



Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
P1	0	7	31	31	24
P2	1	4	9	8	4
P3	2	15	55	53	38
P4	3	11	56	53	42
P5	4	20	66	62	42
P6	4	9	50	46	37

ROUND ROBIN SCHEDULING

- Average Completion Time = (31 +9 + 55 +56 +66 + 50) / 6
- Average Waiting Time = (24+4+38+42+42+37) / 6
- Average Turn Around Time = (31 + 8 + 53 + 53 + 62 + 46) / 6

PRIORITY CPU SCHEDULING

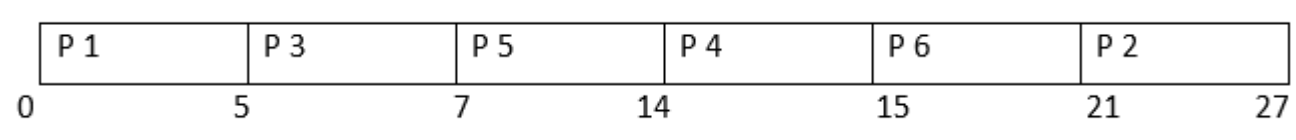
- The Priority CPU Scheduling is different from the remaining CPU Scheduling Algorithms.
- Here, each and every process has a certain Priority number.
- There are two types of priority values
  - Highest number is considered as highest priority value
  - Lowest number is considered as highest priority value
- This can happen in both preemptive strategy and non-preemptive strategy.
- When a task with a lower priority is being performed while a task with a higher priority arrives, the task with the lower priority is replaced by the task with the higher priority, and the latter is stopped until the execution is finished.

PRIORITY CPU SCHEDULING

Process ID	Arrival Time	Burst Time	Priority
P1	0	5	5
P2	1	6	4
P3	2	2	0
P4	3	1	2
P5	4	7	1
P6	4	6	3

Here, in this problem the priority number with highest number is least prioritized. This means 5 has the least priority and 0 has the highest priority.

PRIORITY CPU SCHEDULING-NON PREEMPTIVE APPROACH



Process ID	Arrival Time	Burst Time	Priority	Completion time	Turnaround time	Waiting time
P1	0	5	5	5	5	0
P2	1	6	4	27	26	20
P3	2	2	0	7	5	3
P4	3	1	2	15	12	11
P5	4	7	1	14	10	3
P6	4	6	3	21	17	11

PRIORITY CPU SCHEDULING

- Average Completion Time ?
- Average Waiting Time ?
- Average Turn Around Time ?

THREADS

- A process is divided into number of lightweight process, each light weight subprocess is said to be a Thread.
- Shares memory and resources with other threads of the same process.
- The Thread has a program counter (Keeps track of which instruction to execute next), registers (holds its current working variables), and stack (execution History).
- Intra-process communication is easier and faster.Less context switching overhead.
- Thread States are born State, ready State, running, Sleep , and dead.

Ex-Word processor

Typing, Formatting, spell checking, savings are threads

DIFFERENCE BETWEEN PROCESS AND THREAD

Process	Thread
Process takes more time to create.	Thread takes less time to create.
it takes more time to complete execution & terminate.	Less time to terminate.
Execution is very slow.	Execution is very fast.
It takes more time to switch b/w two processes.	It takes less time to switch b/w two threads.
Communication b/w two processes is difficult .	Communication b/w two threads is easy.
Process can't share the same memory area.	Threads can share same memory area.
System calls are requested to communicate each other.	System calls are not required.
Process is loosely coupled.	Threads are tightly coupled.
It requires more resources to execute.	Requires few resources to execute.

Thank You!